deepin Agent Teams 第一阶段设计文档

1. 项目基本信息

- 赛道：飞桨黑客松第十期 文心合作伙伴赛道
- 厂商：统信
- 任务编号：27
- 任务名称：deepin Agent Teams 智能体团队协作系统
- 代码仓库：https://github.com/guanzi008/deepin_assistant

2. 项目定位

这个项目准备做成一个运行在 `deepin 25` 上的桌面助手。它不是单独的聊天窗口，而是一个放在桌面侧边的统一入口。用户继续按原来的方式使用系统，助手负责读取当前任务上下文，判断用户可能要做什么，再给出草稿、诊断结果或可执行方案。

当前准备聚焦两个固定场景：

1. 智能邮件助手
2. 系统问题智能诊断与修复

这样做的好处是范围清楚，演示路径也稳定，不会把项目做成一个概念很大、落地很弱的通用对话产品。

3. 现有基础

仓库里已经有一套可运行的原型，主要包括：

- 统一图形界面
- 实时系统诊断
- 动作执行控制台
- 支持包导出
- 诊断工单导出
- 高权限动作确认、回执和日志

后续开发会在这套原型上继续收敛，把能力往赛题要求的两个场景上集中。

4. 统一交互入口

项目采用一个固定入口：

- 右侧助手面板

这个面板负责：

- 接收自然语言输入
- 展示系统主动建议
- 展示执行预览
- 请求用户确认
- 展示处理结果

这样符合赛题“唯一用户交互界面”的要求，也更适合录制演示视频。

5. 环境信息采集

项目准备接入的上下文信息包括：

- 当前活动窗口标题、应用名、窗口类名
- 剪贴板内容
- 定向截图和 OCR 结果
- 服务状态与系统日志
- 设备状态、网络状态、软件安装状态
- 与当前任务相关的本地文件摘要

采集策略会尽量克制：

- 先读取窗口标题、剪贴板这类低成本信息
- 只有在需要的时候才做截图
- 对敏感内容的读取和执行动作保留明确确认

6. 角色划分

项目中会固定保留 4 个角色：

6.1 信息收集员

负责采集窗口、剪贴板、截图和系统状态，并整理成统一输入。

6.2 系统操作员

负责判断问题类型、补问缺失信息、生成处理步骤，并调用本地工具。

6.3 内容整理员

负责邮件草稿、结果摘要和说明文本。

6.4 验证员

负责检查动作是否真正生效，并决定是结束还是继续追查。

这样拆分不是为了堆概念，而是为了让“采集、判断、执行、校验”四个环节职责清楚，便于实现和演示。

7. 模型接入方式

赛题要求至少接入两款文心模型 API，项目中会保留两个用途不同的模型：

- 一个负责意图判断和任务路由
- 一个负责邮件整理、诊断推理和结果生成

这样分工更接近实际调用方式，也便于控制延迟和成本。

8. 场景一：智能邮件助手

这个场景的重点，不是单纯生成一段文本，而是根据当前桌面上下文把邮件写作过程真正串起来。

预期流程

1. 用户打开项目文档、issue 页面或会议记录
2. 用户复制关键信息
3. 助手读取当前窗口、剪贴板和相关文件摘要
4. 系统判断当前场景适合生成邮件
5. 输出邮件主题、收件人建议和正文草稿
6. 发送前保留预览和人工确认

场景目标

- 邮件内容来自真实上下文，而不是凭空生成
- 邮件结构完整，可直接修改
- 发送前仍由用户决定

9. 场景二：系统问题智能诊断与修复

这个场景准备覆盖 deepin 桌面上更容易稳定演示的高频问题，而不是只围绕单一打印参数做细枝末节优化。

优先支持的问题

- 打印机连不上
- 没有声音了
- 帮我安装微信
- 网络异常
- 软件源异常

预期流程

1. 用户输入问题
2. 系统判断是否需要补问
3. 收集服务状态、日志、设备状态和安装状态
4. 给出诊断结论和处理方案
5. 用户确认后执行
6. 回读结果并校验是否解决

场景目标

- 诊断依据清楚
- 修复动作可控
- 结果可回读、可解释

10. 技术实现

前端

- React + Vite

- 统一界面：侧边助手面板 + 状态展示 + 执行控制台

本地服务层

- Node.js
- 负责调用系统命令、整理上下文、输出动作预览、执行已确认动作、保存日志和回执

系统工具

- `systemctl`
- `journalctl`
- `lpstat`
- `nmcli`
- `apt`
- 文件检索与日志导出

邮件能力

- 邮件草稿导出
- SMTP 发送测试邮件

11. 隐私与安全

项目会把这部分作为明确边界处理：

- 截图按需触发，不做无差别采集
- 邮件发送前必须确认
- 系统修改前必须确认
- 高权限动作保留日志和执行结果

12. 下一步工作

1. 补齐邮件场景的上下文聚合和草稿生成功能
2. 补齐活动窗口、剪贴板、定向截图等环境采集链路
3. 将系统问题场景扩展到打印、声音、安装和网络等高频问题
4. 在 deepin 25 真机上完成演示验证、截图和录屏

13. 交付内容

按赛题要求，后续会整理以下材料：

- 完整源代码

- 部署说明
- 演示脚本
- 技术报告
- 演示视频

第一阶段先提交这份设计文档；进入锁定名单后，再继续补齐代码、演示和技术报告。

14. 结论

这个项目的重点不是做一个泛化的“AI 演示”，而是做一个放在 deepin 桌面上能实际展示、能说明设计取舍、也能复现的助手。范围上先收敛，先把邮件和系统修复两个场景做完整，再继续扩展。